Boring Chores (chores)

The best thing about *online* programming contests is that you don't need to set up an actual server, you just connect to your *online server* and then install what you need. For *offline* contests, instead, you are often faced with the hard reality: servers break, burst into flames, and you always need to replace parts.

Having prepared a lot of offline contests, William and Giorgio know full well how boring these "chores" can be, so they are now trying to predict how much time will be lost to them. The main problem, is that you don't know in advance where you will need to go: it may be possible that the parts you need are located in *some far away city*, and retrieving them may take a long time.

Furthermore, the contest venue may be decided at the last minute, so we don't even know in advance *which city will host the contest!*

Luckily, Giorgio has a map of all the cities and a selected set of roads (the "best" ones), so that there is exactly one path between any two cities following the map. The roads are bidirectional.

Figure 1: Doing chores can be very tedious.

The contest plan (after the contest city is revealed) is this:

- William will check which parts of the server need to be replaced.
- Then he will go to take the replacement parts.
- Finally, he will come back to the contest city.

Given that we don't know which city will host the contest, nor which city has the server parts we need, help William compute how long it will take him *in the worst case* to go take the server parts and then come back to the contest city.

> ☞ Among the attachments of this task you may find a template file `chores.*` with a sample incomplete implementation.

Input

The first line contains the only integer N. The next $N-1$ lines contain pairs of integers A_i and B_i. Each pair represents a bidirectional road between the A_i-th and B_i-th cities.

Output

You need to write a single line with an integer: the total "number of roads" it will take from the contest venue to the farthest city and come back home (in the worst case).

Constraints

- $1 \leq N \leq 100\,000$.
- $0 \leq A_i, B_i \leq N - 1$ for each $i = 0 \ldots N - 1$.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

- **Subtask 1** [**5 points**]: Examples.
- **Subtask 2** [**20 points**]: The cities lay on a single line of roads.[1]
- **Subtask 3** [**20 points**]: $N \leq 10$.
- **Subtask 4** [**25 points**]: $N \leq 100$.
- **Subtask 5** [**30 points**]: No additional limitations.

Examples

input.txt	output.txt
6 5 3 5 4 0 5 1 5 4 2	6
10 3 9 0 9 2 9 4 7 2 1 2 7 9 6 5 3 2 8	10

Explanation

In the **first sample case** it could happen that the contest venue is on the 3-th city and the parts are in the 2-th city. In this case, William would need to cross 6 roads $(3 - 5 - 4 - 2 - 4 - 5 - 3)$.

In the **second sample case** it could happen to have the contest in the 4-th city and the server parts in the 5-th city.

[1] In other words, each city is connected with at most 2 other cities.